

Academic OS

Student & Admin Authentication

Implementation documentation — login, session, and route protection

STUDENT LOGIN

ADMIN LOGIN

ROUTE PROTECTION

Codebase: bros-code (DSA/SQL code-judge LMS pilot)

Covers PRD Feature 4 (Student Authentication), Feature 5 (Admin Authentication),
and the credential-issuing half of Feature 11 (Student Management)

Student & Admin Authentication

Architecture, data model, and request flow for the login system

1. Why this exists

The pilot has a fixed, known cohort of 100 students and 1–2 admin/content-ops staff. Per the PRD, there is **no self-registration for either role** — students sign in with a Student ID and password issued by an admin; admins sign in with an email and password provisioned outside the app. This document describes the login, session, and route-protection system built to satisfy that requirement, plus the admin-side "Student Management" screen that issues student credentials in bulk, since there is no student self-signup flow to build instead.

2. Architecture at a glance

1. **One User table**, distinguished by a `role` enum (`ADMIN` / `STUDENT`) rather than separate tables — keeps the session shape simple and matches the single `{userId, role}` contract every existing route already depended on.
2. **Stateless signed session** — a JWT (HS256, via `jose`) is issued on login and stored in an `httpOnly`, `SameSite=lax` cookie. No server-side session table; the token itself is the source of truth, verified on every request.
3. **Two-layer protection** — `src/proxy.ts` does an optimistic, pre-render redirect for logged-out visitors; every Server Component, Route Handler, and Server Action independently re-verifies the session before doing anything sensitive. Proxy is never trusted as the sole gate.
4. **Passwords hashed with bcrypt** (12 salt rounds) — never stored or logged in plaintext. Student passwords are auto-generated and shown to the admin exactly once at creation time.

3. Data model

```
enum Role { ADMIN STUDENT }

model User {
  id          String   @id @default(cuid())
  role        Role
  email       String?  @unique // admin login identifier
  studentId   String?  @unique // e.g. "2026-CSE-0417", student login identifier
  name        String
  passwordHash String
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}
```

`email` and `studentId` are both nullable and unique — Postgres allows multiple `NULL` s in a unique column, so an admin row (no `studentId`) and a student row (no `email`) coexist without a partial index. Which field is required for which role is enforced in application code (Zod schemas + the login/creation logic), not as a database constraint.

`Problem.createdBy` and `Submission.userId` — previously loose, unconstrained strings — are now real foreign keys into `User`, so every problem and submission is traceably owned by a real account.

4. Session lifecycle

Login

1. Student or admin submits credentials via `/login` or `/admin/login`.
2. A Server Action validates the input (Zod), looks up the `User` by `studentId` or `email`, and compares the password against `passwordHash` with `bcrypt.compare`.
3. On success, a JWT is signed with payload `{ sub: userId, role, name, email? }` and a 7-day expiry.
4. The token is set as an `httpOnly`, `Secure` (in production), `SameSite=lax` cookie named `session`.
5. The user is redirected — students to `/`, admins to `/admin/problems`.

On failure, both portals show one generic message ("that ID/email or password doesn't match") without indicating which field was wrong, per the PRD's error-state requirement.

Every subsequent request

Proxy (`src/proxy.ts`) reads the `session` cookie, verifies the JWT, and redirects logged-out visitors to the correct login page before any page renders. Already-authenticated users hitting a login page are bounced to their landing page instead.

Per-route check — Server Components, Route Handlers, and Server Actions each call `getSessionUserFromCookies()` or `getSessionUser(req)` (`src/lib/session.ts`) independently and re-check `role` before doing anything sensitive. This mirrors the codebase's pre-existing convention and follows the Next.js guidance that Proxy alone must never be the only authorization check, since Server Actions are POSTs to their own route and can silently bypass a Proxy matcher exclusion.

Logout

A shared Server Action (`src/app/actions/logout.ts`) deletes the `session` cookie. Two thin wrappers, `logoutStudent` and `logoutAdmin`, redirect back to each portal's own login page.

5. Route protection matrix

Path	Unauthenticated	Wrong role	Correct role
<code>/login</code> , <code>/admin/login</code>	Renders normally	Bounced to own landing page if already logged in as the other role	Bounced to own landing page if already logged in
<code>/</code> , <code>/problems/[slug]</code>	Redirect → <code>/login</code>	N/A (both roles may browse)	Renders
<code>/admin/**</code>	Redirect → <code>/admin/login</code>	Logged in as student → 403 Forbidden page (not bounced to a login form they can't use)	Renders

`/api/**`

Excluded from Proxy's matcher — each Route Handler independently checks `getSessionUser(req)` and returns 401/403 JSON on failure.

Design note: for admin routes, an unauthenticated visitor is redirected to `/admin/login`, but a visitor who *is* logged in — just with the wrong role — sees a Forbidden page instead of being redirected to a login form they already can't use. This split is implemented across two layers: Proxy only redirects when there is *no* session at all; the admin layout (`src/app/admin/(protected)/layout.tsx`) handles the wrong-role case.

6. Student credential issuance (Student Management)

Since students cannot self-register, admins bulk-create accounts at `/admin/students`. This is the "registration" counterpart to student login.

1. Admin enters one student per line (`studentId, name`).
2. The Server Action pre-validates every Student ID for uniqueness — both within the batch and against existing rows — *before* creating anything, so a single duplicate can't leave a partial batch of already-generated, already-shown passwords behind.
3. A random password is generated per student (excluding visually ambiguous characters like `0 / 0` or `1 / l`, since these are hand-transcribed to a student, not copy-pasted), hashed with `bcrypt`, and the account is created.
4. The generated plaintext passwords are returned to the admin's browser and rendered *once*, with a clear "won't be shown again" warning. They are never stored or logged in plaintext, and are not recoverable after this point — only a further admin-initiated reset would issue a new one.

7. Files added or changed

File	Purpose
<code>prisma/schema.prisma</code>	<code>User</code> model, <code>Role</code> enum, FKs on <code>Problem / Submission</code>
<code>src/lib/password.ts</code>	<code>bcrypt</code> hash/verify, random password generation
<code>src/lib/jwt.ts</code>	JWT sign/verify, cookie name & options
<code>src/lib/session.ts</code>	Rewritten — same <code>getSessionUser / getSessionUserFromCookies</code> contract, real verification internally
<code>src/lib/authSchema.ts</code>	Zod schemas for both login forms and bulk student creation
<code>src/app/login/page.tsx</code>	Student login
<code>src/app/admin/login/page.tsx</code>	Admin login
<code>src/app/actions/logout.ts</code>	Shared logout, per-portal redirect
<code>src/app/admin/(protected)/layout.tsx</code>	Centralized admin auth gate + logout header (moved from <code>admin/layout.tsx</code>)

<code>src/app/admin/(protected)/students/*</code>	Student Management: list + bulk-add with one-time password reveal
<code>src/proxy.ts</code>	Optimistic route-level redirect (Next.js 16's renamed <code>middleware.ts</code>)
<code>src/app/api/problems/[id]/submissions/route.ts</code>	Now derives <code>userId</code> from the session instead of trusting the request body
<code>prisma/seed.ts</code>	Bootstraps the first admin account from <code>ADMIN_EMAIL</code> / <code>ADMIN_PASSWORD</code>

8. Environment variables

Variable	Purpose
<code>SESSION_SECRET</code>	Signing key for session JWTs — long random string, e.g. <code>openssl rand -base64 32</code>
<code>ADMIN_EMAIL</code> / <code>ADMIN_PASSWORD</code>	Bootstrap credentials for the first admin account, consumed once by <code>npm run db:seed</code>

9. Verification performed

- Prisma migrations applied cleanly against local Postgres in two steps (add `User` / `Role` , then add FKs after a backfill pass) — no data loss on existing seeded problems.
- `next build` and `eslint` both pass clean.
- End-to-end, over HTTP against a running dev server: admin login succeeds and sets a correctly-shaped session cookie; wrong password is rejected with a generic error and no cookie; an authenticated admin session reaches `/admin/problems` and `/admin/students` ; a logged-out visitor is redirected from `/` , `/admin/problems` ; a logged-in student hitting `/admin/problems` sees a 403 Forbidden page rather than a login redirect; already-authenticated visitors hitting either login page bounce to their landing page; logout clears the session.
- A student account was created via the same write path as the bulk-add action, logged in successfully, and used to submit a real solution — confirmed the resulting `Submission.userId` resolves via the new foreign key to that student's account, and that an unauthenticated submission attempt is now rejected with `401` (previously it silently trusted a client-supplied `userId`).

Generated as part of the Academic OS authentication implementation.